Estimate Ridge

Estimate ridge function for jammer removal

```
def _odd(n):
    """
    Returns n if odd, n-1, otherwise
    """
    return n if n % 2 else n-1


def estimate_ridge(X, meta, band=(1600, 3400), fs=Fs):
    """
    X is array of STFTs
    Dominant frequency per frame inside `band`
    (parabolic interpolation on log-mag), then median + Savitzky-Golay
smoothing.
    Estimate is from DFT X of mixture (jammed) signal x[n].
    """
    f = frame_freqs(meta)   # bin frequencies in Hz
    sel = np.where((f >= band[0]) & (f <= band[1]))[0]    # selection indexes
    mag = np.abs(X[:,sel])    # shortcut for magnitudes in selected band
    k = np.argmax(mag, axis=1)    # indexes of largest magnitudes versus time
    peak = mag[np.arange(len(k)), k]   # largest magnitude for each frame
    raw = f[sel][k].copy()
    km = np.clip (k, 1, len(sel)-2)
    a, b, c = (np.log(mag[np.arange(len(k)), km + d] + 1e-30) for d in (-1,
0, 1))
    den = a - 2*b + c
    off = np.where(np.abs(den) > 1e-12, 0.5*(a-c)/den, 0.0)
    raw = f[sel][km] + np.clip(off, -1, 1) * (f[1]-f[0])
    # frames with no real energy in band (pure padding) carry no information:
interpolate
    if np.any(peak > 0):
        valid = peak > 0.02*np.median(peak[peak>0])
    else:
        np.zeros_like(peak, bool)
    m = np.arange(len(raw))
    if valid.sum() >= 2:
        raw = np.interp(m, m[valid], raw[valid])
    M = len(raw)
    if M >= 3:
        r = ss.medfilt(raw, _odd(min(7, M)))
    else:
        r = ss.medfilt(raw, 1)
    wl = _odd(min(21, M))
    if wl >=5:
        r = ss.savgol_filter(r, wl, 2, mode="interp")
```

```
    return r, raw, peak
```

This function, `estimate_ridge()`, is designed to estimate the instantaneous frequency trajectory of a strong narrowband signal, such as a frequency-varying jammer, from the Short-Time Fourier Transform (STFT) of a signal.

It does this by finding the largest spectral peak in each STFT frame, improving the frequency estimate using parabolic interpolation, and smoothing the resulting frequency trajectory.

I'll go through the DSP concepts, explain the Python operations, and point out a few potential problems in the implementation.

1. What the function is trying to accomplish

Suppose the received signal is

\[x[n]=s[n]+j[n]+w[n] \]

where:

- \(s[n]\) is the desired signal.
- \(j[n]\) is a narrowband interference signal or jammer.
- \(w[n]\) is noise.

Suppose the jammer has a frequency that changes with time:

\[j[n]=A\cos\left(2\pi \frac{f_j[n]}{f_s}n+\phi\right) \]

Strictly speaking, for a time-varying instantaneous frequency, the phase should be accumulated over time rather than simply multiplying \(f_j[n]\) by \(n\). But the important point is that the jammer produces a spectral peak that moves between STFT frames.

For example:

Illustrative STFT of a frequency-varying jammer

iframe

The bright band represents a moving spectral peak. The dashed white line illustrates the frequency ridge that the function attempts to follow. Synthetic example.

The function tracks that ridge, producing one estimated frequency for each STFT frame.

2. Function definition

```
def estimate_ridge(X, meta, band=(1600, 3400), fs=Fs):
```

The arguments are:

Argument	Meaning
`X`	Two-dimensional array containing the STFT
`meta`	Information about how the STFT was calculated
`band`	Frequency search interval, default 1600–3400 Hz
`fs`	Sampling frequency

The code assumes that `X` has shape

```
(number_of_frames, number_of_frequency_bins)
```

Notice that `fs` is never actually used inside the function. Presumably, the sampling frequency is already contained in `meta` and used by `frame_freqs(meta)`.

3. Find the frequency bins in the search band

```
f = frame_freqs(meta)sel = np.where((f >= band[0]) & (f <= band[1]))[0]
```

The first statement determines the actual frequency in Hz corresponding to each FFT bin.

For an \(N\)-point DFT, assuming an ordinary nonnegative-frequency ordering,

\[f_k=\frac{k f_s}{N} \]

and the bin spacing is

\[\Delta f=\frac{f_s}{N}. \]

The second statement creates an array of indices corresponding to frequencies between 1600 and 3400 Hz.

For example, suppose

```
f = np.array([0, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000])
```

Then

```
sel = np.where((f >= 1600) & (f <= 3400))[0]
```

produces

```
array([4, 5, 6])
```

These indices correspond to 2000, 2500, and 3000 Hz.

The `[0]` extracts the index array from the tuple returned by `np.where()`.

4. Find the strongest spectral peak in each STFT frame

```
mag = np.abs(X[:,sel])k = np.argmax(mag, axis=1)peak = mag[np.arange(len(k)),
k]raw = f[sel][k].copy()
```

This is an important part of the function.

First statement

```
mag = np.abs(X[:,sel])
```

The STFT is generally complex:

\[X[m,k]=\sum_n x[n]w[n-mH]e^{-j2\pi kn/N} \]

where \(m\) is the frame index, \(k\) is the frequency-bin index, and \(H\) is the hop size.

Taking the magnitude gives

\[M[m,k]=|X[m,k]|. \]

`X[:,sel]` selects all time frames but only the frequency bins inside the desired band.

Second statement

```
k = np.argmax(mag, axis=1)
```

This finds the frequency-bin index having the largest magnitude in each time frame.

The option `axis=1` means to search across the frequency bins, rather than across time.

Mathematically,

\[k_{\max}[m]=\underset{k}{\operatorname{argmax}}\;M[m,k]. \]

For example:

```
mag = np.array([    [1, 7, 3, 2],    [2, 4, 9, 1],    [8, 3, 2, 1]])k =
np.argmax(mag, axis=1)
```

produces

```
array([1, 2, 0])
```

The dominant spectral bins are therefore bins 1, 2, and 0 for the three frames.

Third statement

```
peak = mag[np.arange(len(k)), k]
```

This retrieves the actual magnitude of the peak for each frame.

In the example, `peak` becomes

```
array([7, 9, 8])
```

The two index arrays work together:

```
np.arange(len(k))   # [0, 1, 2]  row indicesk                    # [1, 2, 0]
column indices
```

Consequently, NumPy retrieves

```
mag[0,1], mag[1,2], mag[2,0]
```

This is called advanced indexing in NumPy.

Fourth statement

```
raw = f[sel][k].copy()
```

This converts the peak bin indices into frequencies in Hz.

At this point, the frequency estimates are quantized to DFT-bin centers.

For example, if the FFT has frequency spacing of 20 Hz, a true sinusoidal frequency of 2437 Hz might initially be estimated as 2440 Hz.

The next part of the function attempts to improve this accuracy.

5. Parabolic interpolation: estimating frequencies between FFT bins

```
km = np.clip(k, 1, len(sel)-2)a, b, c = (np.log(mag[np.arange(len(k)), km + d] + 1e-30)           for d in (-1, 0, 1))den = a - 2*b + coff = np.where(np.abs(den) > 1e-12,               0.5*(a-c)/den, 0.0)raw = f[sel][km] + np.clip(off, -1, 1) * (f[1]-f[0])
```

This is arguably the most interesting DSP calculation in the function.

The FFT estimates the frequency spectrum only at discrete frequency-bin centers, but the spectral maximum of a sinusoid will generally fall between bins.

One way to estimate the actual location of the maximum is to fit a parabola through three adjacent spectral samples.

Why three samples?

Suppose we have the logarithmic magnitudes

$$ a=\log M[k-1],\qquad b=\log M[k],\qquad c=\log M[k+1]. $$

We fit a second-order polynomial of the form

$$ p(\delta)=A\delta^2+B\delta+C $$

through the points

$$ (-1,a),\quad (0,b),\quad (1,c). $$

Solving gives

$$ A=\frac{a-2b+c}{2}, \qquad B=\frac{c-a}{2},\qquad C=b. $$

The vertex of the parabola occurs where its derivative is zero:

$$ p'(\delta)=2A\delta+B=0. $$

Therefore,

$$ \boxed{\delta=\frac12\frac{a-c}{a-2b+c}} $$

which is exactly the formula implemented in the Python code.

Interactive illustration of parabolic peak interpolation

Sub-bin peak offset

0.35 bins

a

-5.81

b

-2.07

c

-2.73

Estimated frequency: 2407.0 Hz, assuming bin k is 2400 Hz and bin spacing is 20 Hz.

What `np.clip()` does

```
km = np.clip(k, 1, len(sel)-2)
```

The interpolation requires the bins immediately to the left and right of the peak. This statement prevents array indexing outside the selected frequency range.

For example, a peak originally located at bin 0 is moved to bin 1 for interpolation.

However, this also means that the interpolation is no longer centered on the original maximum for peaks at either band edge.

Why use the logarithm?

```
np.log(mag[...] + 1e-30)
```

The spectral peak of a windowed sinusoid can often be approximated reasonably well by a parabola in the log-magnitude domain, particularly near its maximum.

The small constant `1e-30` prevents attempting to compute $\log(0)$.

For a Gaussian window, the log-magnitude spectral peak has a particularly natural parabolic shape. Other windows, including Hann and rectangular windows, produce approximation errors.

Convert back to Hz

The final interpolated frequency is

$$\boxed{\hat f[m]=(k_{\max}[m]+\delta[m])\Delta f}$$

with the appropriate frequency-bin indexing offset.

The implementation calculates the same thing with:

```
raw = f[sel][km] + np.clip(off, -1, 1) * (f[1]-f[0])
```

For example, with a 20 Hz FFT-bin spacing, a 2400 Hz peak-bin center and an interpolated offset of 0.35 bins give

$$\hat f=2400+0.35(20)=2407 \text{Hz}.$$

One subtle issue: the code clips offsets to ±1 bin. For an interior local maximum with well-behaved neighboring samples, an offset between approximately −0.5 and +0.5 bins is generally expected.

6. Identify frames without a reliable spectral peak

```
if np.any(peak > 0):    valid = peak > 0.02*np.median(peak[peak>0])else:
np.zeros_like(peak, bool)
```

Some STFT frames may contain almost no signal energy, particularly at the beginning or end of a recording where padding was added.

In these cases, the estimated peak frequency has very little meaning.

This code classifies a frame as valid when its peak magnitude exceeds 2% of the median nonzero peak magnitude:

$$\text{valid}[m]= \begin{cases} \text{True}, & P[m]>0.02P_{\mathrm{median}}\\ \text{False},&\text{otherwise}. \end{cases}$$

Here, $P[m]$ is the maximum spectral magnitude in the frequency band for frame m.

Notice the distinction between `peak` and `raw`:

- `peak` represents the strength of the spectral maximum.
- `raw` represents its estimated frequency.

There is a programming error in the `else` branch.

```
else:    np.zeros_like(peak, bool)
```

This creates a Boolean array but does not assign it to `valid`.

The corrected code should be:

```
else:    valid = np.zeros_like(peak, dtype=bool)
```

Without this correction, an all-zero peak array produces an `UnboundLocalError` when the code reaches `valid.sum()`.

Also, this threshold is only a relative amplitude test. It does not establish that a peak is genuinely a jammer. Broadband noise or a strong desired signal can also produce a peak exceeding this threshold.

7. Interpolate missing frequency estimates

```
m = np.arange(len(raw))if valid.sum() >= 2:    raw = np.interp(m, m[valid], raw[valid])
```

This step attempts to fill in frequency estimates for frames where the energy was too small to produce a reliable estimate.

Suppose we have:

Frame	Original estimated frequency	Valid?
0	2100 Hz	Yes
1	2150 Hz	Yes
2	1800 Hz	No
3	1900 Hz	No
4	2300 Hz	Yes

Linear interpolation replaces the two invalid values with approximately 2200 and 2250 Hz.

This assumes that the jammer frequency changes smoothly between the valid frames.

Importantly, `np.interp()` holds the nearest valid value constant outside the range of valid frames. It does not extrapolate the observed slope.

8. Median filtering

```
M = len(raw)if M >= 3:    r = ss.medfilt(raw, _odd(min(7, M)))else:    r = ss.medfilt(raw, 1)
```

This applies a median filter to the sequence of frequency estimates.

The function `_odd()` presumably returns an odd integer, because median-filter window lengths must be odd.

For most sufficiently long recordings, the median filter uses seven frames.

A median filter is effective at removing isolated outliers. Suppose the ridge estimate is

```
[2000, 2010, 2020, 3150, 2040, 2050, 2060]
```

The 3150 Hz estimate is probably an erroneous peak picked from some other spectral component.

A seven-point median filter replaces that isolated value with 2040 Hz, the median of the seven values.

Unlike a moving-average filter, the median filter can eliminate a large isolated error without allowing that error to pull the surrounding estimates toward it.

One detail worth recognizing is that `scipy.signal.medfilt()` uses zero padding at the sequence boundaries. This can influence the first and last few frequency estimates, especially for short sequences.

9. Savitzky–Golay smoothing

```
wl = _odd(min(21, M))if wl >=5:    r = ss.savgol_filter(r, wl, 2, mode="interp")
```

After removing isolated outliers, the function performs additional smoothing using a Savitzky–Golay filter.

For most recordings, this filter uses:

- A 21-frame window.
- A second-degree polynomial.
- Boundary handling using polynomial interpolation.

The idea is to fit a local quadratic polynomial to the ridge:

\[\hat f[m+\ell]\approx a_0+a_1\ell+a_2\ell^2. \]

The smoothed value at the center is the fitted polynomial evaluated at \(\ell=0\), which is \(a_0\).

The principal advantage over ordinary moving-average filtering is that Savitzky–Golay smoothing can preserve slowly varying slopes and curvature.

For a frequency-swept jammer, this is useful because we want to remove random fluctuations without flattening the true frequency trajectory excessively.

The median filter and Savitzky–Golay filter therefore perform different tasks:

Filter	Main purpose
Median, 7 frames	Remove isolated incorrect frequency estimates
Savitzky–Golay, 21 frames	Smooth the remaining frequency trajectory while preserving gradual variations

There is a tradeoff, however. If the jammer changes frequency rapidly, a 21-frame window may smooth away real variations.

The appropriate filter lengths depend on the STFT hop time. For example, a 21-frame window spans roughly 210 ms if the hop time is 10 ms, but only about 21 ms if the hop time is 1 ms.

10. Returned values

```
return r, raw, peak
```

The function returns three arrays.

Return value	Description
`r`	Final smoothed estimate of the frequency ridge, in Hz
`raw`	Parabolically interpolated frequency estimate, with low-energy frames filled by linear interpolation
`peak`	Magnitude of the largest spectral component in the selected band, before frequency interpolation

Each array has one element per STFT frame.

The function could therefore be called as:

```
ridge, raw_freq, strength = estimate_ridge(X, meta)
```

The ridge might subsequently be used to track and remove frequency-varying interference.

11. Overall signal-processing flow

STFT \(X[m,k]\)

Complex time-frequency array

Band selection and magnitude

Restrict to 1600–3400 Hz

Peak detection

Find maximum-magnitude bin in each frame

Parabolic interpolation

Estimate frequency between FFT bins

Low-energy frame interpolation

Replace unreliable frequency estimates

Median and Savitzky–Golay filtering

Remove spikes and smooth frequency trajectory

Estimated jammer ridge \(\hat f[m]\)

One estimated frequency per STFT frame

12. Important limitations and improvements

There are several things I would pay attention to in using this function for narrowband jammer tracking.

First, it does not really track the jammer from one frame to the next. It independently finds the largest spectral component in each frame and then smooths the results. If the desired signal produces a stronger peak than the jammer in some frames, the ridge can jump to the desired signal. Median filtering may correct isolated errors, but not extended periods of incorrect peak selection.

Second, the all-zero-input bug should be fixed. The `else` branch must assign an array to `valid`.

Third, at least three selected frequency bins are needed. Otherwise, the parabolic interpolation indexing is not valid. It would be sensible to check this explicitly.

Fourth, zero padding provides more interpolation samples, but not more inherent frequency resolution. If the STFT was calculated using a heavily zero-padded FFT, neighboring bins are strongly correlated. Parabolic interpolation still works as a numerical peak-location estimator, but its accuracy depends on the window, signal-to-noise ratio, and zero-padding factor. A larger FFT does not by itself improve the ability to resolve two closely spaced sinusoids.

Fifth, the code assumes that the frequencies returned by `frame_freqs(meta)` are uniformly spaced and increasing. This is typical for a one-sided STFT but should be verified.

Sixth, a jammer with abrupt frequency jumps may be handled poorly. The median filter can eliminate short real changes, and Savitzky–Golay smoothing can spread a discontinuity across multiple frames.

The central DSP idea

The most important point is that this is a spectral peak tracking algorithm operating on the time-frequency representation of a signal.

It estimates

\[\boxed{\hat f[m]\approx \underset{f\in[1600,3400]}{\operatorname{argmax}}\;|X(m,f)|} \]

with sub-bin interpolation and temporal smoothing.

It is not computing instantaneous frequency directly from the complex phase of the signal, nor is it performing a statistical maximum-likelihood frequency estimate. It is assuming

that the signal of interest corresponds to the strongest spectral component within a predefined frequency band.

For your application involving the removal of a nonstationary jamming tone, that is a reasonable starting approach when the jammer is strong and spectrally isolated. The next major improvement would be to incorporate continuity of the estimated frequency from one frame to the next, rather than always selecting the independently strongest peak in every frame.